SIH 2026 PS 26262: Person 1 Work Pack

Role	**Person 1: Approval workflow and EVC**
Repository	`approval-workflow-ocr-compliance`, branch `person-1`
Team	Person 1 (Workflow + EVC) · Person 2 (OCR + Compliance + Deadlines) · Person 3 (Frontend + Dashboards + Security + Audit)
Full plan	`SIH26262_Team_Split.pdf` (everything, all three people)

Contents

1. Your part
2. What is already built for you
3. Rules that prevent conflicts
4. Interfaces between people
5. Where your work meets the others
6. Your week-by-week plan
7. Your demo steps
8. Branches and integration
9. Day 0 checklist (shared)

---

1. Your part: Approval workflow and EVC

This pack contains everything Person 1 needs day to day. Section references such as "§19" point to the final research report.

Mission. An application enters, follows the correct Table 2.3 route for its pinned ruleset version, moves only through valid transitions, and reaches EVC with conflict-free dates.

1.1 Owns

Area	Details	Report
Intake	`institution`, `application`, `submission`; category checklist (Annexure-2); DSC flag; "can't submit with mandatory items missing"	§17.1
Route plan	On submit, return the route (e.g. Scrutiny → EVC → SHC as applicable) with clause citation	§4.2, §18
Rule registry	Loads, versions and pins rulesets (`aph-2024-27@r1`); static validation; approval flow for new rulesets; `r3-DEMO` overlay; rule-change impact replay (Tier 2)	§18
`routes.yaml`	All 21 Table 2.3 routes plus NEW_INSTITUTION (Chapter I); SHC "as applicable" exactly where EVC = Yes	§4.2
State machine	`route_def`, `ruleset`, `transition`, `stage_visit`; guards; reverts with reasons; 100% of invalid transitions rejected	§18
Deficiency loop	`deficiency`, `deficiency_item`; Re-Scrutiny scope = open items only; items traceable to responses	§4.4
Committee workspace (backend)	`committee`, `task`, `observation`; append-only report versions attributed to members	§17.1

Area	Details	Report
EVC	`evc_request`, `expert`, `availability`, `coi`, `slot`, `assignment`; labelled stand-in "authorised selector" for panels (APH 1.8(a): automated, **not random**); CP-SAT chooses **dates only**; EDF baseline; "why this slot" explanations; propose/publish	§23
Letter issuance trigger	On LoA/EoA/LoD issue, call P2's `register_letter`	§6.2
Mock adapters	AICTE portal events, NSWS (labelled SIMULATED); inbound dedup	§17
Synthetic case generator	~300 institutions, 9 ROs, ~600 applications in the Table 2.3 mix, ≥5 seeds; planted workflow events (deficiency loops, late responses, EVC delays); writes the answer key	§28.2
Tier 2 extras	BPMN view of route versions; cited case summaries (local LLM, optional)	§28.1
Spike	Operaton 2-day decision spike (alternative workflow core)	§18

1.2 APIs owned

- `POST /applications` · `POST /applications/{id}/submit`
- `POST /cases/{id}/actions` (all transitions; guarded and idempotent through P3's middleware)
- `GET /evc/queue` · `POST /evc/schedule:propose` · `POST /evc/schedule:publish`
- `POST /admin/rulesets` · `POST /admin/rulesets/{id}/validate` · `POST /admin/rulesets/{id}/approve`

1.3 Calls other people's ports

Calls	Owner	When
`deadlines.open / close / pause_internal / resume_internal`	P2	Every transition whose YAML has `timers:` (e.g. record_deficiency pauses AICTE_PROCESSING and starts INSTITUTION_RESPONSE)
`compliance.register_letter`	P2	LoA/EoA/LoD/corrigendum issued
`ledger.append`, `notes.create`, `authz.check`, `uow`	P3	Every command

1.4 Acceptance tests (P1 proves these)

- Each of the 21 routes plus NEW_INSTITUTION produces the stage list in §4.2. This is a table-driven test straight from the YAML.
- 100% of invalid transitions are rejected with a reason code.
- A case pinned to r1 stays on r1 after r3-DEMO is loaded, and the rule-version switch shows the difference (demo step 10).
- Re-Scrutiny shows only open items.
- The scheduler reports 0 hard-constraint violations (COI, availability, no expert double-booked) and beats EDF on the same queue.

1.5 Open questions P1 chases (§34)

- Q1: official APH URL plus amendments. The Dec 2025 corrigendum stays r3-DEMO only until it is confirmed.
- Q2: academic calendar EVC and LoA dates.
- Q3: Annexure-7 flow chart and the full Annexure-2 lists per category.
- Q5: number of Regional Offices.

No APH source was found for EVC duration or for the deficiency-response window. Both remain configurable demo assumptions in YAML, labelled as such.

---

2. What is already built for you

The starter project is already in the repository, so nobody starts from an empty folder. It proves the three parts are genuinely connected: **an institution submits an application in the UI → Person 1's workflow picks the route → Person 2's engine opens a clock → Person 3's ledger records it and the UI shows the state and audit timeline.**

Verified on 24 Oct 2026:

- **Tests:** 31 automated tests pass (unit + end-to-end).
- **Docker:** the full flow was run through the real API in Docker.
- **UI:** the UI was checked against it in a browser.

Person 1: already in place

Done	Where
Ruleset registry: loads, validates and pins `aph-2024-27@r1`; refuses `r3-DEMO` outside demo mode	`backend/app/workflow/registry.py`
All 21 Table 2.3 routes + NEW_INSTITUTION, with labels; static check that SHC appears exactly where EVC does	`rulesets/aph-2024-27/r1/routes.yaml`
`clocks.on_submit` in YAML decides which clocks a submission opens (no hard-coding)	`routes.yaml`
Institutions, applications, stage visits; create + submit in one transaction	`backend/app/intake/`
APIs: `GET /institutions`, `POST /applications`, `POST /applications/{id}/submit`, `GET /applications`, `GET /rulesets`, `GET /rulesets/{key}`	`intake/api.py`, `workflow/api.py`
3 fictional DEMO institutions	`generator/cases/seed_demo.py`
Tests: all routes, corrected rows, static validation, overlay refusal	`backend/tests/test_rulesets.py`

Your next steps:

1. The state machine and transitions from `routes.yaml`, starting with `record_deficiency` and `submit_compliance`. They call `DeadlinePort.pause_internal / resume_internal`, which already exist.
2. The deficiency loop with items.
3. The committee backend.
4. The EVC queue.
5. The scale generator and answer key.

---

3. The three rules that prevent conflicts

Rule 1: one writer per table. Each table belongs to exactly one person's PostgreSQL schema (`wf`, `evc`, `doc`, `cmp`, `dl`, `plat`, `an`). Code only *writes* its own schema. To change someone else's data, call their port function (section 6), never an UPDATE on their table. Reads across schemas go through read-only views that the owner publishes.

Rule 2: one owner per folder. The folder-to-person table is in `docs/OWNERSHIP.md`, and it uses only "Person 1/2/3". `CODEOWNERS` lists all three GitHub usernames for every path, so GitHub asks the team for review without naming who owns which part. Folder ownership is therefore a **written team rule**: change someone else's folder only with their agreement. `contracts/` needs approval from all three. CODEOWNERS accepts any one owner's approval, so a separate CI check (`contracts-approval`) enforces the three-way approval.

```
approval-workflow-ocr-compliance/
  contracts/                    ALL THREE (CI blocks merge until both other members approve)
    ports.py                    Python Protocols modules call each other through
    types.py, enums.py          shared value types and vocabularies (roles, stages, states, prefixes)
    errors.py                   errors any module may raise (mapped to HTTP codes)
    fakes.py                    in-memory stand-in for every port
    events/<prefix>/<name>.v<N>.json   JSON Schema per event; no free-text fields allowed
    openapi.json                snapshot of the real API; CI fails if it drifts
  backend/app/
    platform/                   P3  unit of work, demo auth + policy, ledger + verify, notes,
                                    personal data, outbox (next: Keycloak, Cerbos, notifier, idempotency)
    intake/                     P1  institutions, applications, submission
    workflow/                   P1  ruleset registry (next: state machine, deficiency loop, committees)
    evc/                        P1  queue, expert pool, COI, selector stand-in, CP-SAT scheduler
    documents/                  P2  upload, ClamAV, router, OCR engines, extractor, facts
    compliance/                 P2  approval letters, obligations, evidence, verification
    deadlines/                  P2  rules, due-date math, engine, clock (next: calendars, timer, alerts)
    analytics/                  P3  projections, dashboards, reports, scorecard
    main.py                     P3  discovers each module's models.py, wiring.py and api.py
  backend/migrations/versions/<module>/   each owner: one Alembic chain per module (Rule 3)
  rulesets/aph-2024-27/r1/routes.yaml       P1
  rulesets/aph-2024-27/r1/deadlines.yaml    P2
  rulesets/aph-2024-27/r3-DEMO/overlay.yaml P1 (deadline keys need P2 approval)
  calendars/                    P2
  generator/cases/              P1  (seed_demo.py today; scale generator next)
  generator/documents/          P2  (also the OCR evaluation set)
  frontend/                     P3
  docker-compose.yml, backend/Dockerfile, .github/, scripts/   P3
  tests/e2e/stepNN_*_test.py  owner of each demo step (section 12): one file each
  docs/                         ARCHITECTURE, OWNERSHIP, TEAM_WORKFLOW
```

Rule 3: no shared migration head. Each module keeps its own Alembic chain in `backend/migrations/versions/<module>/` with its own `branch_labels`, so three people never edit the same revision file. Upgrade with `alembic -c backend/alembic.ini upgrade heads` (plural). `scripts/check_migration_heads.py` runs in CI and fails if any module accidentally grows a second head. Cross-schema foreign keys are allowed only to the stable `id` of a table owned by someone else, and that owner promises never to rename or drop it.

Rule 4: one command, one transaction. A workflow transition, the deadlines it opens, its ledger events and any outbox rows commit atomically in PostgreSQL (`unit_of_work`). There is no asynchronous event processing for mandatory work such as creating a deadline. Only notification **delivery** happens later: Person 3's notifier reads the outbox after commit.

Supporting habits:

- **Event names carry the module prefix:** `wf.*`, `evc.*`, `doc.*`, `cmp.*`, `dl.*`, `plat.*`. Two people therefore cannot invent the same event name.
- **Stubs first.** Every port has a fake in `contracts/fakes.py`, so nobody waits on anyone.
- **Person branches, frequent integration.** Each person works on their own `person-N` branch. Branches are integrated into `main` every 2–3 days (section 14).
- **Lightweight local setup.** `docker compose up` runs only Postgres, the backend and the frontend. Heavier services (Keycloak, Cerbos, ClamAV, MinIO) come later under an optional profile.
- **A 15-minute daily sync** covers only what is blocked and which contract needs to change.

---

4. Interfaces between people (the contracts)

These are the signatures now in `contracts/ports.py`. A port is a Python call inside the same process and the same database transaction, not HTTP. This keeps "transition + deadlines + ledger" atomic. Each module registers its implementations in its own `wiring.py`, and callers look them up with `port(PortType)`.

```
# contracts/ports.py  (all three approve changes)

class UnitOfWork(Protocol):                     # P3 - one transaction per command
    session: Session
    principal: Principal

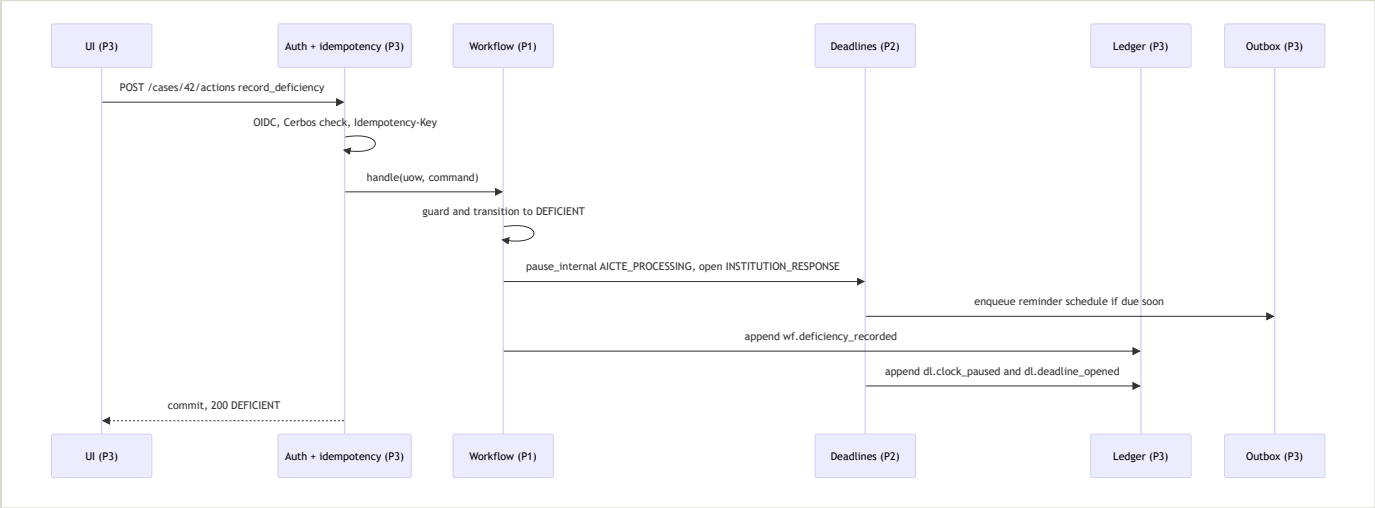
# ---- provided by P3 (platform) ----
class LedgerPort(Protocol):
    def append(self, uow, event: EventIn) -> int: ...           # returns seq; schema-checked, hash-chaine
    def verify(self, session) -> VerifyResult: ...
class NotesPort(Protocol):
    def create(self, uow, body: str) -> tuple[UUID, str]: ...   # (note_ref, note_hash)
class PersonalDataPort(Protocol):
    def store(self, uow, data: dict, purpose: str) -> UUID: ...
class OutboxPort(Protocol):
    def enqueue(self, uow, msg: OutboundMessage) -> None: ...
class AuthzPort(Protocol):
    def check(self, principal, action: str, resource: ResourceRef) -> None: ...

# ---- provided by P1 (workflow / intake) ----
class RulesetPort(Protocol):
    def default_key(self) -> str: ...                           # "aph-2024-27@r1"
    def get(self, key: str) -> RuleSet: ...                     # routes + parsed deadline rules
    def route_for(self, category: str, key: str) -> Route: ...
class CasePort(Protocol):
    def lookup(self, session, application_id: UUID) -> CaseRef: ...   # institution + region, for access c

# ---- provided by P2 (deadlines / compliance) ----
class DeadlineRuleParserPort(Protocol):
    def parse(self, raw: dict) -> dict: ...                     # P1's registry loads deadlines.yaml throu
class DeadlinePort(Protocol):
    def open(self, uow, subject: SubjectRef, rule_id: str, anchor: Anchor, ruleset_key: str) -> UUID: ...
    def close(self, uow, subject: SubjectRef, rule_id: str, outcome: str) -> None: ...
    def pause_internal(self, uow, subject: SubjectRef, rule_id: str) -> None: ...   # LEGAL clocks refuse
    def resume_internal(self, uow, subject: SubjectRef, rule_id: str) -> None: ...
class CompliancePort(Protocol):
    def register_letter(self, uow, letter: LetterIn) -> UUID: ...
class ClockPort(Protocol):
    def now(self) -> datetime: ...                              # the ONLY source of time (time travel)
```

Shared value types (`contracts/types.py`): `Principal`, `SubjectRef(type, id)`, `ResourceRef`, `CaseRef`, `Anchor(at, event_seq)`, `EventIn(aggregate_type, aggregate_id, event_type, payload, event_version, reason_code, ruleset_rev, note_ref, note_hash, personal_ref)`, `VerifyResult`, `OutboundMessage(alert_id, channel, template, recipient_ref)`, `LetterIn(...)`, `RouteStage`, `Route`, `RuleSet`. Errors (`contracts/errors.py`): `ValidationFailed` 422, `NotFound` 404, `Forbidden` 403, `Conflict` 409.

4.1 One command crossing all three people



If any step fails, the whole transaction rolls back. No half-done state exists, so there is nothing for two people to argue about.

4.2 EoA with deficiencies (P1 → P2 → P3)

1. P1: transition `issue_eoa` calls `compliance.register_letter(letter_type=EOA, lists_deficiencies=true, issue_date=portal date)`.
2. P2: creates the letter row. The officer confirms the conditions, which creates obligations with `state_origin=OFFICER_CONFIRMED`. `deadlines.open(rule EOA_DEFICIENCY_6M, anchor=letter.issue_date)` runs.
3. P2: at the T-30 / T-7 / T-1 reminder levels (§4.4) the timer creates `alert` rows and calls `outbox.enqueue`.
4. P3: the notifier delivers, retries and logs `delivery` rows. The UI shows the alert. The ack goes to P2's endpoint, which stops escalation.

4.3 Lease with OCR (P2 → P3 → P2)

1. P2: upload → ClamAV → router → OCR → `fact` PROPOSED (or a handwriting flag).
2. P3: review screen. The officer types the date, then the suggestion is revealed, then the officer confirms or corrects (calls P2's `POST /facts/{id}/confirm`).
3. P2: the confirmed fact opens the lease deadline (2.1(e) stop-AY) and the 18-month reminder.

---

5. Where your work meets the others

These are the places where two people could both think something is theirs. Each has one owner.

Overlap point	Owner	The other person's role
Rule registry vs `deadlines.yaml`	P1 owns the registry, loader and versioning	P2 owns the deadline file contents and its validator function
`r3-DEMO` overlay	P1	P2 approves if it touches deadline keys
Approval letter: issuing it vs storing it	P1 triggers the issue	P2 owns the `approval_letter` row and everything that follows
EVC overdue escalation	P2 (it is a deadline)	P1 opens the EVC clock
Pendency numbers	P3 computes the projection	P1 owns `stage_visit` events; P2 owns INTERNAL clocks
Free-text reasons	P3 `case_note`	P1/P2 call `notes.create`, never put text in payloads
Personal data (names, phones)	P3 `personal_data`	P1/P2 store a reference only
Synthetic data	P1 cases + EVC demand; P2 documents + OCR set	P3 consumes the answer key in the scorecard
Cerbos policies	P3 writes the policy files	P1/P2 list actions and resource attributes per endpoint
Idempotency	P3 middleware	P1/P2 make handlers deterministic
Inbound adapter dedup	Adapter owner (P1 portal/NSWS, P2 DigiLocker)	Use P3's idempotency table
Institution master data	P1	P3 uses `institution_id` for scoping
User/role master data	P3	P1 references `expert.user_id`

---

6. Your week-by-week plan

Checkpoints come from §28.3: W1 events end to end · W2 scorecard green · W4 Tier 2 behind flags · W6 freeze and rehearsal.

Week	P1
0 (Day 0)	Contracts (section 7)
1	Registry + r1 routes (all 21) + state machine; submit returns route
2	Deficiency loop + committee backend; case generator + answer key
3	EVC queue, expert pool, COI, selector stub; EDF baseline
4	CP-SAT scheduler + explanations; rule-change impact replay; r3-DEMO switch
5	BPMN view; NSWS/portal mocks; Operaton spike write-up; hardening
6	Bug fixes; demo steps 1–4, 10

---

7. Your demo steps

The demo script is §29. Each step has one owner, who also owns its e2e test file.

Step	What happens	Owner
1	Sunrise Institute submits; route from `aph-2024-27@r1` with citation	P1
2	2 deficiencies; AICTE clock pauses, institution clock starts	P1 (clocks shown by P2)

Step	What happens	Owner
3	Re-Scrutiny on open items only; advance to EVC queue	P1
4	CP-SAT dates for 40 EVCs, 0 conflicts vs EDF; panels from the stand-in selector	P1
10	"Why is this due?" then the r1 vs r3-DEMO switch	P1 (panel by P3)

Questions from evaluators (§31.1) go to the owner of the area: rules and EVC to P1; OCR, deadlines and law-based dates to P2; security, privacy, audit and data to P3.

---

8. Branches and integration

```
main            always runs; changed only by integration
 ├── person-1   Person 1 works here
 ├── person-2   Person 2 works here
 └── person-3   Person 3 works here
```

Every day:

```
git switch person-2                 # your own branch
# ...code...
python -m pytest                    # must pass before you push
git add -A
git commit -m "Add escalation ladder"
git push
```

Every 2–3 days: integration.

1. All three person branches are merged into a short-lived `integration/YYYY-MM-DD` branch, in this order:
   - `contracts/`;
   - Person 3's platform;
   - Persons 1 and 2.
2. Conflicts are resolved and every check runs: tests, the contract check, migration heads, and a frontend build.
3. Anything a person changed **outside their own folders** is flagged and discussed before merging.
4. A pull request goes to `main`. Another team member approves it, and it merges.

Right after each integration, everyone syncs:

```
git switch person-1
git pull origin main
git push
```

Why integrate every 2–3 days and not only at the end:

- Two branches can merge without a conflict and still break the app, for example a function called in one shape and written in another.
- Small integrations catch this early, while the change is still fresh.
- Months of divergence surface only in demo week.

Commit messages: short, imperative, about the change (`Add deficiency loop transitions`, `Fix month-end clamp`).

Repository settings (repo owner, once): under Settings → Branches, add a rule for `main`:

- Require a pull request with 1 approval and Code Owner review.
- Require the `backend`, `frontend` and `all-three-approve` checks.
- Disallow bypassing these settings.

Private repositories need GitHub Pro for this; GitHub Education gives students Pro for free.

---

9. Day 0: what all three do together

Status on 24 Oct 2026: items 1–5, 7 and 8 are **done** and pushed. Items 6 and 9 are still open and should be done together at the first team meeting.

#	Deliverable	Lead	Status
1	Repo, CODEOWNERS, CI, `docker compose up`	P3	Done
2	`ports.py` + value types (section 6)	All	Done; **all three must read and agree before building on it**
3	`enums.py`: roles, stages, states, event prefixes	All	Done (reason codes added per module as needed)
4	Event schemas	Each for own module	Done for Week 1 (`wf.application_created`, `wf.application_submitted`, `dl.deadline_opened`); add yours as you build
5	API contract	Each for own endpoints	Done as `contracts/openapi.json`, generated from the real API; CI fails if it drifts
6	Answer-key schema	P1 + P3 (P2 adds OCR fields)	**Open**
7	Ruleset YAML format	P1 + P2	Done: r1 routes (all 22) and deadlines validate
8	Fake implementation of every port	Each for own ports	Done (`contracts/fakes.py`, checked by CI)
9	Wireframes for each screen	P1/P2 sketch, P3 approves	**Open**

Rule for changing a contract later: 1. Edit `contracts/` and update `contracts/fakes.py` in the same branch. 2. If an API changed, run `python scripts/check_contracts.py --write` to refresh `contracts/openapi.json`. 3. Explain the change at integration. Both other members approve; CI enforces it.